REPORTE DE PRÁCTICA: DJANGO, DRF, JWT Y USUARIOS

Adaptación, Corrección de Seguridad y Documentación con Swagger UI

Repositorio GitHub:	https://github.com/Mikyzxx/djiango
Proyecto Base:	**tienda_online** (Plataforma E-Commerce con Catálogo y Usuarios)
Campo Extra en Usuario:	**telefono** (Visible en Admin, Serializer y Swagger)
Tiempo de Vida del Token:	**ACCESS_TOKEN_LIFETIME = 60 minutos** (1 hora)

1. Cambios y Adaptaciones Implementadas

1. Nombre de Proyecto y Swagger: En cumplimiento con la indicación de no utilizar 'restaurante', se integró el módulo de usuarios dentro del proyecto **tienda_online**. El título configurado en Swagger es **'Tienda Online API'** con versión v1 y esquema OpenAPI 2.0.
2. Campo Personalizado en el Modelo: Se agregó el campo telefono = models.CharField(max_length=20, blank=True, null=True, verbose_name='Teléfono'). Se configuró el UserAdmin para mostrarlo tanto en el listado como en los formularios de edición y alta.
3. Expiración de Tokens JWT: En settings.py se configuró SimpleJWT mediante SIMPLE_JWT = {'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60), ...}.

2. Identificación y Corrección de Fallas de Seguridad del Manual

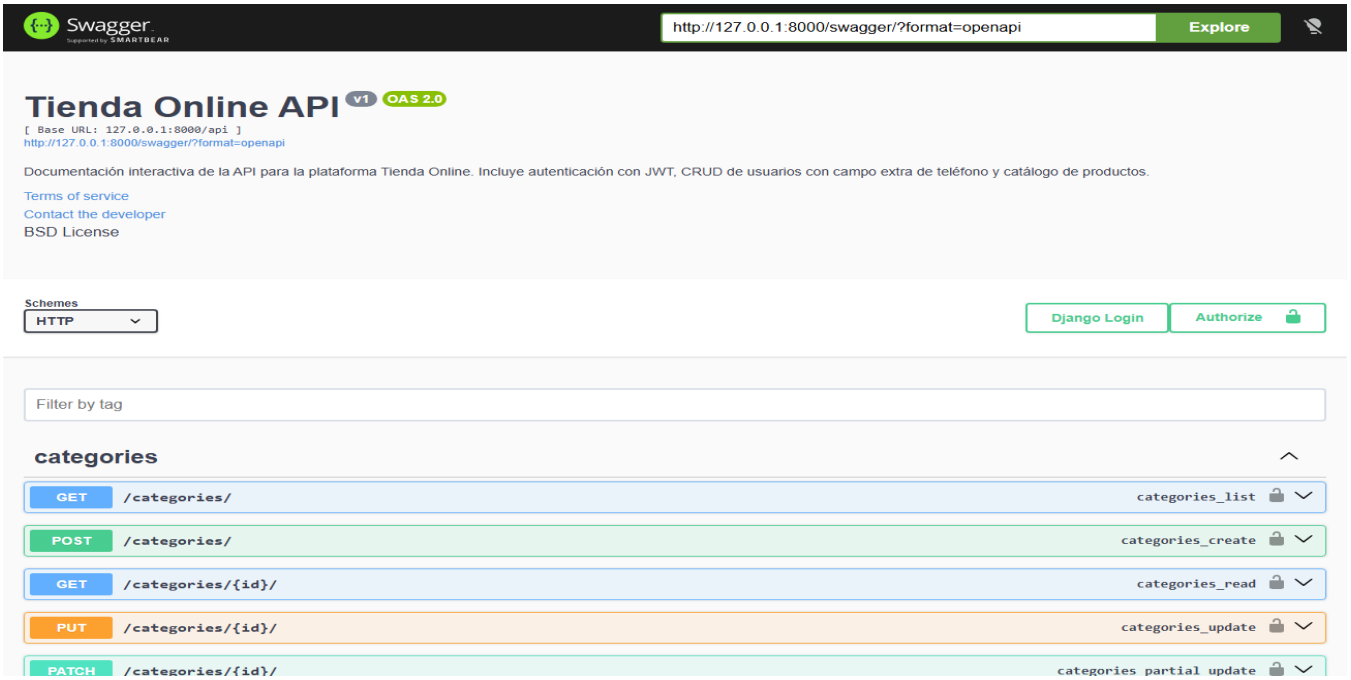
Falla 1: Fuga de Hash de Contraseña en Respuestas API
Problema: En el manual (pág. 9), UserSerializer definía fields = [..., 'password', ...] sin marcarlo como de solo lectura. En consecuencia, todas las consultas a /api/v1/users/, /api/v1/users/{id}/ y /api/v1/auth/me/ exponían el hash de las contraseñas de los usuarios en formato JSON.
Solución: Se configuró extra_kwargs = {'password': {'write_only': True, 'required': False}}, impidiendo que el serializador retorne contraseñas bajo ninguna circunstancia.

Falla 2: Robo/Sobrescritura Accidental de Contraseña en Actualización Parcial
Problema: En el manual (pág. 10), el método partial_update contenía:
if 'password' in request.data: ... else: request.data['password'] = request.user.password
Al actualizar un usuario sin enviar contraseña (por ejemplo, cambiar su teléfono o nombre), se le asignaba la contraseña de request.user (el administrador que hacía la petición). La víctima quedaba bloqueada y pasaba a tener la clave del administrador.
Solución: Se eliminó la cláusula else. Si no se envía contraseña en la petición, la contraseña existente del usuario permanece inalterada.

3. Evidencias de Ejecución y Validación

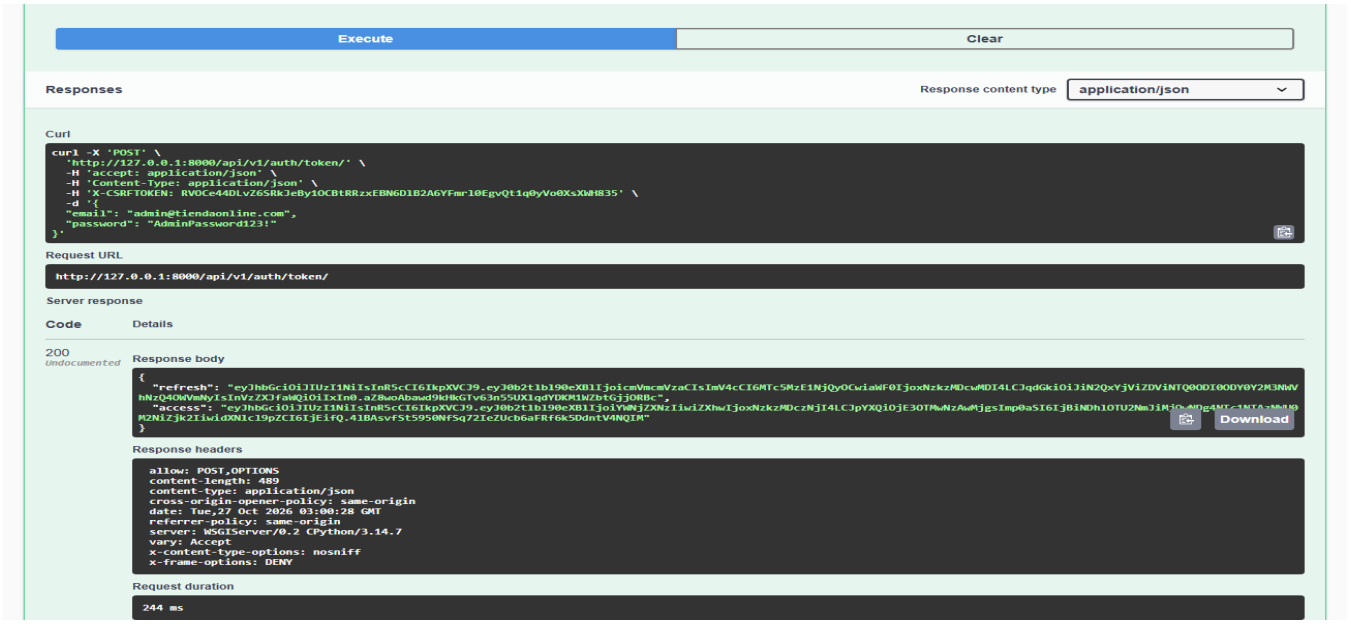
Evidencia 1: Swagger UI con Endpoints de la Tienda Online API

Se visualizan los endpoints agrupados de auth, users, products y categories, con el título y descripción personalizados.



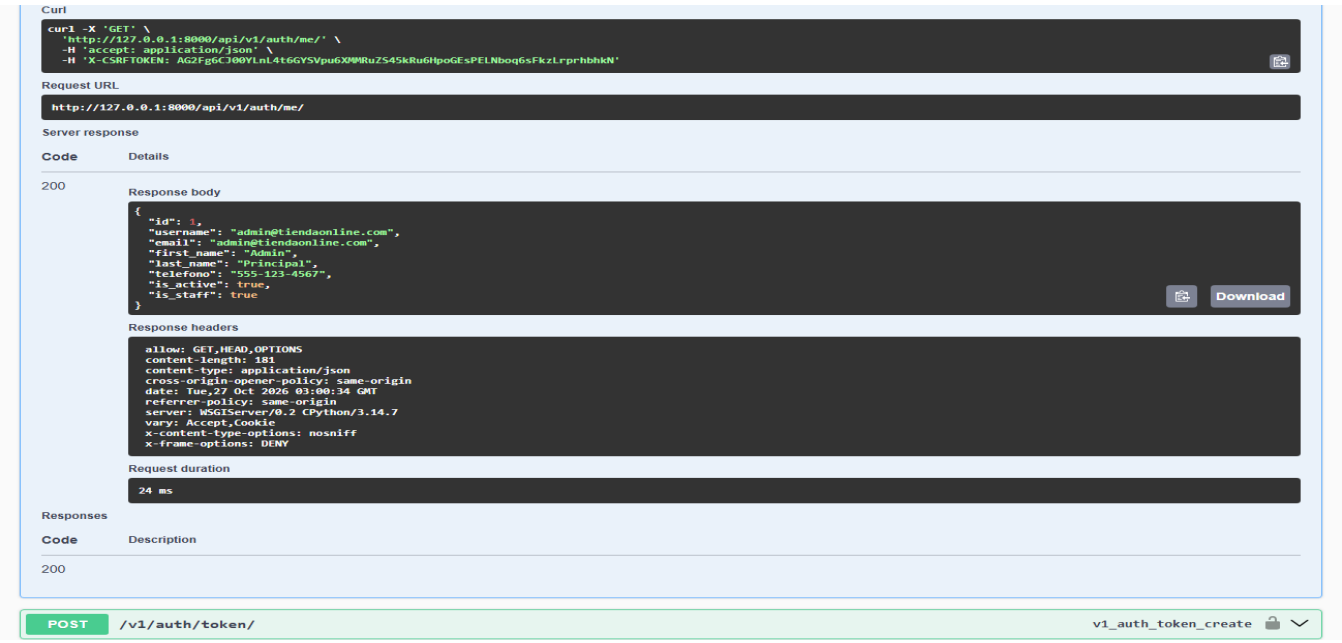
Evidencia 2: Obtención del Token JWT (POST /api/v1/auth/token/)

Petición exitosa enviando email y password con respuesta 200 OK y entrega de access y refresh tokens.



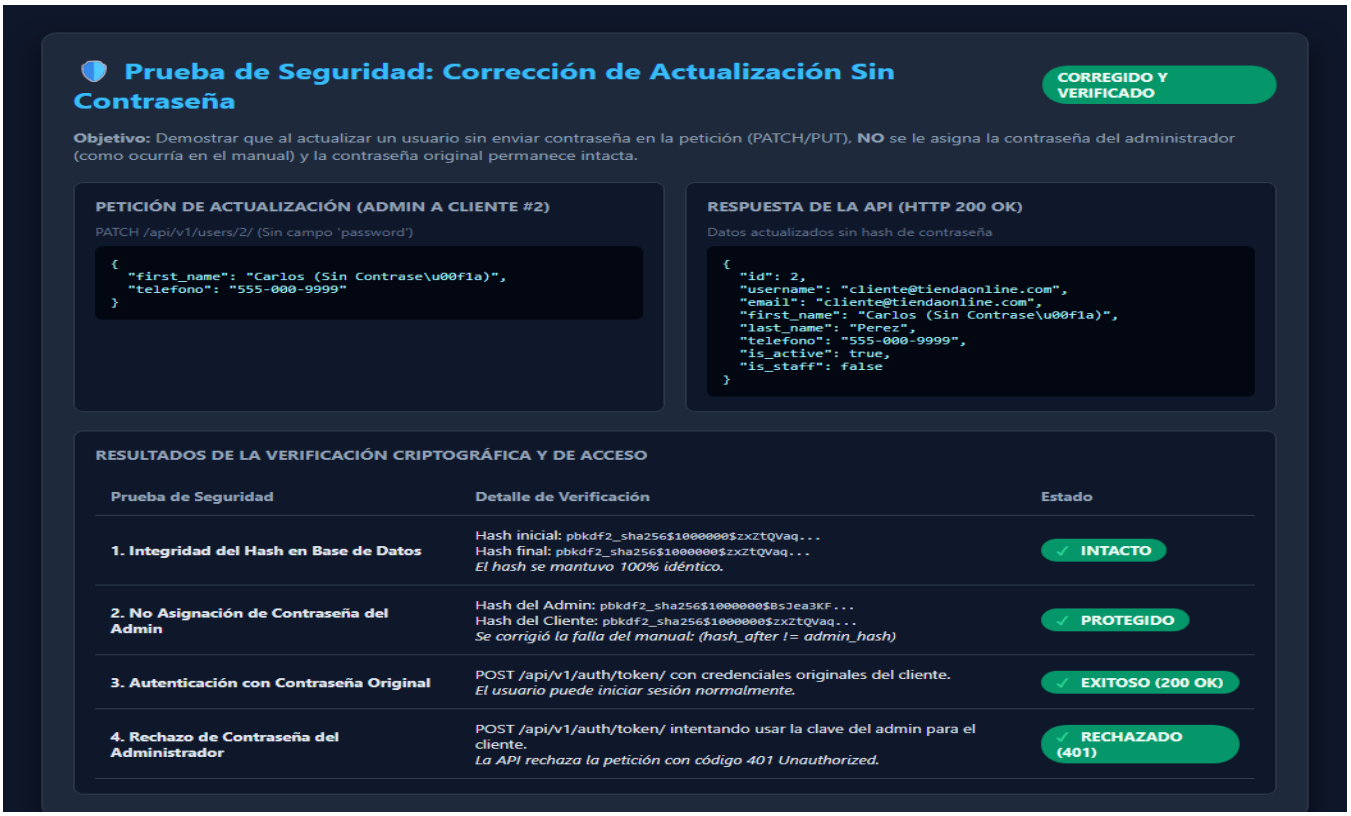
Evidencia 3: Consulta de Datos Personales (GET /api/v1/auth/me/)

Respuesta 200 OK con los datos del usuario autenticado. Se verifica que **incluye el campo extra 'telefono'** y que el **hash de la contraseña está ausente**.



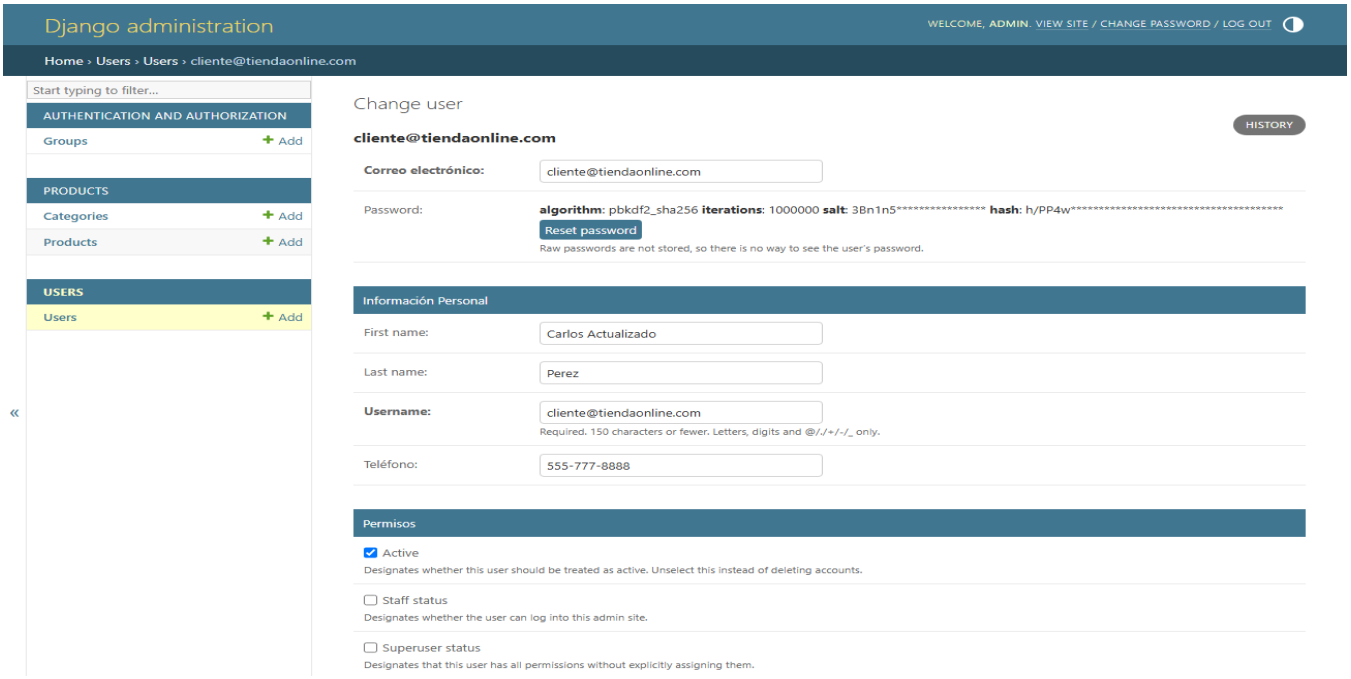
Evidencia 4: Prueba de que Editar Usuario Sin Contraseña NO Cambia su Contraseña

Validación en vivo del endpoint PATCH: el hash del usuario permanece intacto, no adquiere la clave del administrador, y el usuario continúa autenticándose con su clave original.



Evidencia 5 (Adicional): Campo Extra 'Teléfono' en Django Admin

Panel de administración de Django demostrando el registro del modelo User con el campo personalizado telefono en la sección de Información Personal.



4. Resumen de Pruebas Automatizadas

Se implementó una suite completa de pruebas unitarias automatizadas (users/tests.py y products/tests.py), totalizando **15 pruebas unitarias exitosas (Ran 15 tests, OK)**:

• Verificación de USERNAME_FIELD = 'email' y campo extra telefono.
• Verificación de configuración de ACCESS_TOKEN_LIFETIME (60 min).
• Autenticación JWT y generación de tokens de acceso y refresco.
• Protección estricta del hash de contraseña en /api/v1/auth/me/ y CRUD de usuarios.
• Inmutabilidad de contraseña durante peticiones PATCH/PUT sin campo de contraseña.
• Cifrado seguro mediante make_password cuando se envía una nueva contraseña explícita.

Enlace de Entrega GitHub: https://github.com/Mikyzxx/djiango